

MongoDB Reference Guide

Databases, collections, and CRUD operations — definitions, syntax, and examples.

1. Database Operations

Create Database

MongoDB does not have a dedicated command to create a database ahead of time. Switching to a database name that doesn't exist yet (with the use statement) selects it, but the database is only actually created once you insert data into a collection within it.

Syntax

```
use <databaseName>
```

Example

```
use schoolDB
// The database "schoolDB" is now selected, but does not
// exist on disk until a collection is written to it.
db.students.insertOne({ name: "Aditi", grade: 8 });
// "schoolDB" now exists.
```

Use Database

The use statement switches the current database context for all subsequent commands in the session. If the named database does not exist, MongoDB will create it as soon as data is written to it.

Syntax

```
use <databaseName>
```

Example

```
use schoolDB
// All following commands now run against schoolDB
```

Delete Database

`db.dropDatabase()` permanently deletes the currently selected database, including all of its collections and documents. This action cannot be undone.

Syntax

```
db.dropDatabase()
```

Example

```
use schoolDB
db.dropDatabase();
```

```
// schoolDB and all its data are permanently removed
```

2. Collection Operations

Create Collection

`db.createCollection()` explicitly creates a new collection within the current database. This is optional for most use cases, since MongoDB automatically creates a collection the first time you insert a document into it — but explicit creation lets you set options like validation rules or a capped size.

Syntax

```
db.createCollection(<name>, <options>)
```

Example

```
use schoolDB
db.createCollection("students");
// Creates an empty "students" collection
```

Delete Collection

`db.collection.drop()` permanently removes a collection and all of the documents it contains, along with any indexes defined on it. This action cannot be undone.

Syntax

```
db.<collectionName>.drop()
```

Example

```
db.students.drop();
// The "students" collection and all its documents are removed
```

3. CRUD Operations on a Collection

Create — `insertOne()`

Inserts a single document into a collection. If the collection doesn't exist yet, MongoDB creates it automatically.

Syntax

```
db.<collectionName>.insertOne(<document>)
```

Example

```
db.students.insertOne({
```

```
name: "Rahul",
grade: 9,
subjects: ["Math", "Science"]
});
```

Create — insertMany()

Inserts multiple documents into a collection in a single call, passed as an array.

Syntax

```
db.<collectionName>.insertMany([<document1>, <document2>, ...])
```

Example

```
db.students.insertMany([
  { name: "Meera", grade: 7 },
  { name: "Kabir", grade: 8 }
]);
```

Read — find()

Retrieves all documents in a collection that match a given query filter, returning a cursor. Calling find() with no arguments returns every document in the collection.

Syntax

```
db.<collectionName>.find(<query>, <projection>)
```

Example

```
db.students.find({ grade: 8 });
// Returns all documents where grade equals 8
```

Read — findOne()

Retrieves the first document that matches a given query filter, returning a single document (or null) instead of a cursor.

Syntax

```
db.<collectionName>.findOne(<query>, <projection>)
```

Example

```
db.students.findOne({ name: "Rahul" });
// Returns the first matching document, or null if none match
```

Update — updateOne()

Updates the first document that matches a query filter, using update operators such as \$set to modify specific fields without replacing the whole document.

Syntax

```
db.<collectionName>.updateOne(<filter>, <update>)
```

Example

```
db.students.updateOne(
  { name: "Rahul" },
  { $set: { grade: 10 } }
);
```

Update — updateMany()

Updates every document that matches a query filter, applying the same update operators to each matching document.

Syntax

```
db.<collectionName>.updateMany(<filter>, <update>)
```

Example

```
db.students.updateMany(
  { grade: 8 },
  { $set: { promoted: true } }
);
```

Delete — deleteOne()

Deletes the first document that matches a query filter.

Syntax

```
db.<collectionName>.deleteOne(<filter>)
```

Example

```
db.students.deleteOne({ name: "Kabir" });
```

Delete — deleteMany()

Deletes every document that matches a query filter. Passing an empty filter ({}) deletes all documents in the collection, without dropping the collection itself.

Syntax

```
db.<collectionName>.deleteMany(<filter>)
```

Example

```
db.students.deleteMany({ grade: 7 });  
// Removes every document where grade equals 7
```

4. More About find()

find() with a Query Filter

A query filter narrows results to documents matching specific field values or query operators (e.g. \$gt, \$lt, \$in). An empty filter ({}) matches every document.

Syntax

```
db.<collectionName>.find({ <field>: <value> })
```

Example

```
db.students.find({ grade: { $gte: 8 } });  
// Returns students with grade 8 or higher
```

find() with Projection

The second argument to find() is a projection, which controls which fields are included or excluded in the returned documents. Use 1 to include a field and 0 to exclude it.

Syntax

```
db.<collectionName>.find(<query>, { <field>: 1 })
```

Example

```
db.students.find({}, { name: 1, grade: 1, _id: 0 });  
// Returns only the name and grade fields for every student
```

find() with sort()

sort() orders the results of a find() query. Use 1 for ascending order and -1 for descending order on the given field.

Syntax

```
db.<collectionName>.find(<query>).sort({ <field>: 1 | -1 })
```

Example

```
db.students.find().sort({ grade: -1 });  
// Returns students ordered from highest to lowest grade
```

find() with limit() and skip()

limit() caps the number of documents returned by a query. skip() omits a given number of documents from the start of the results — commonly combined for pagination.

Syntax

```
db.<collectionName>.find(<query>).skip(<n>).limit(<n>)
```

Example

```
db.students.find().sort({ grade: 1 }).skip(10).limit(5);  
// Skips the first 10 results, then returns the next 5
```

findOne() Revisited

findOne() behaves like find() but always returns a single document (the first match) rather than a cursor, and does not support cursor methods like sort() or limit() directly.

Syntax

```
db.<collectionName>.findOne(<query>, <projection>)
```

Example

```
db.students.findOne({ grade: { $gte: 8 } }, { name: 1, _id: 0 });  
// Returns the name of the first matching student found
```